

MOTORX

System Explanation and Improvements

A detailed guide in simple language

27 August–27 September 2026

How the system works • What changed • Evidence and remaining issues

Based on repository code, Git history, documentation, and saved test evidence. Test results describe recorded runs; no new application test run was performed for this document.

Contents

1. What MotorX does	4
2. The three main user groups	4
3. How the system is organised	4
4. How signing in and permissions work	6
5. How someone becomes a dealer	6
6. How vehicle listings work	7
7. How dealers add and maintain vehicles	7
8. How bulk CSV uploads work	8
9. How bulk photo uploads work	8
10. How buyers find vehicles	9
11. Vehicle comparison and recommendations	9
12. What administrators do	9
13. Notifications and email	10
14. How MotorX stores information	10
15. What already existed before August 27	11
16. Improvement: dealer application and profile workflows	11
17. Improvement: listing management	11
18. Improvement: older inventory can be reconfirmed	12
19. Improvement: comparison, suggestions, and languages	12
20. Improvement: database duplicate protection	12
21. Improvement: import recovery	13
22. Improvement: safer file handling	13
23. Improvement: abuse controls	14

24. Improvement: stronger moderation	14
25. Improvement: tests are protected from real data	14
26. Improvement: broader automated quality checks	15
27. Improvement: deployment and operational support	15
28. Improvement: photo migration and thumbnails	16
29. Timeline of recorded improvements	17
30. What the saved test evidence proves	17
31. What remains incomplete or needs careful wording	18
32. Plain-language presentation explanation	18

MotorX is a vehicle marketplace that connects buyers with dealers. Buyers can find and compare vehicles, dealers can manage their stock, and administrators can review dealers and moderate the marketplace.

This explanation is based on the application code, documentation, Git history, and saved test results reviewed on September 27, 2026. The latest commit before the review period is from August 23, which was used as the starting point for identifying improvements. Recorded changes in the period begin on September 21. Git dates show when work was committed, not necessarily when development started.

1. What MotorX does

MotorX helps solve three practical problems:

- Buyers need a convenient way to find suitable vehicles and see the seller’s details.
- Dealers need an efficient way to publish and maintain many vehicle advertisements.
- Administrators need tools to control who can sell and handle unsuitable listings or accounts.

For example, a dealer with 200 vehicles should not have to enter every vehicle manually. MotorX lets them upload a spreadsheet containing those vehicles, checks the information, creates valid listings, and reports the rows that need correction.

A buyer can search for something such as “automatic car under 10 million,” inspect matching vehicles, compare options, and view the dealer’s contact information.

The implemented application focuses on vehicle discovery, dealer inventory, and marketplace administration. An implemented checkout or payment workflow was not found in the reviewed application routes.

2. The three main user groups

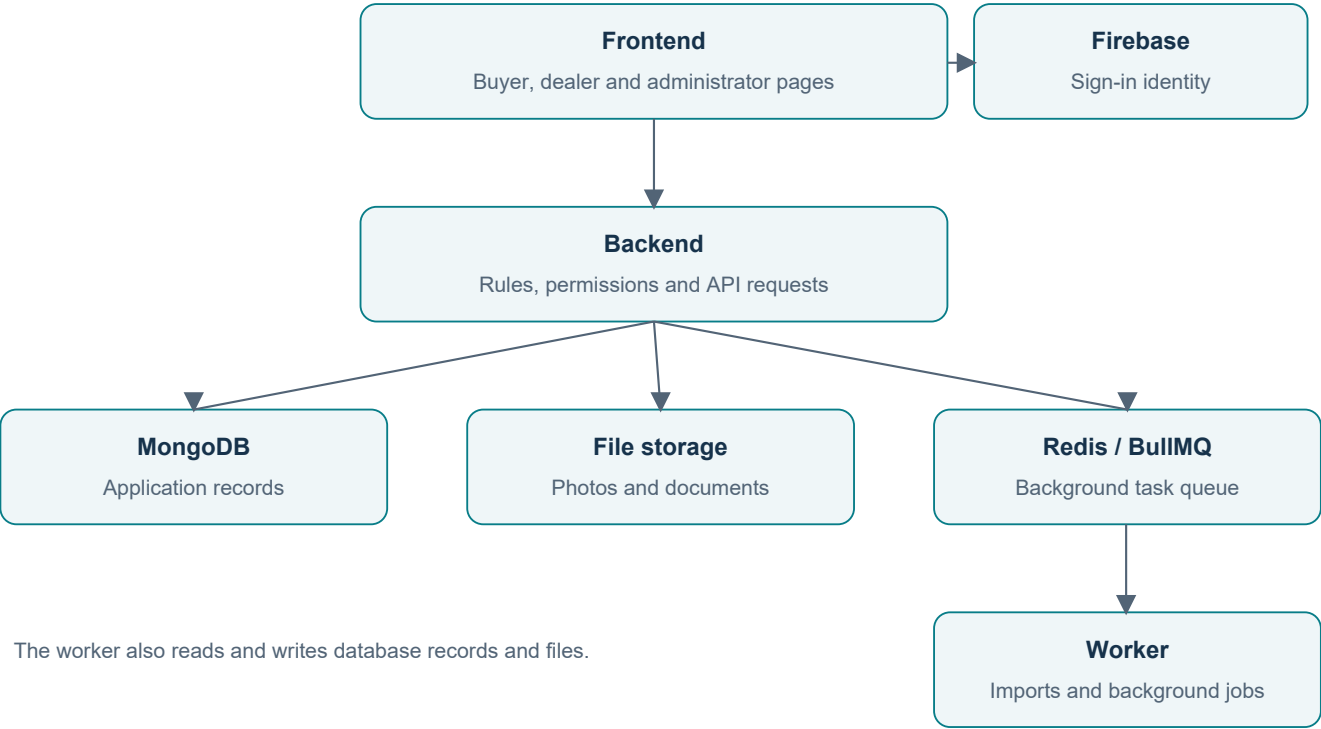
User group	Main purpose	Main capabilities
Buyer or visitor	Find a suitable vehicle	Browse, search, filter, inspect details, compare vehicles, and see suggestions
Dealer	Advertise and maintain vehicle stock	Apply for approval, maintain a business profile, create listings, upload inventory and photos, and update availability
Administrator	Manage the marketplace	Review applications, manage accounts, moderate listings, monitor uploads, and inspect audit records

Public marketplace pages can be opened without signing in. Dealer and administrator operations require authentication and the appropriate permission.

Permissions are checked by the backend. Hiding a button in the browser is not the system’s only protection.

3. How the system is organised

MotorX has several parts, each with a different job.



Part	Technology used	Simple explanation
Frontend	React and TypeScript	The website people see and interact with
Backend	Node.js and Express	Checks requests, applies business rules, and controls access
Database	MongoDB Atlas	Stores users, dealers, listings, upload records, and other application data
Authentication	Firebase Authentication	Confirms who is signing in
Queue	Redis and BullMQ	Holds background tasks until a worker processes them
Worker	Separate Node.js application	Handles time-consuming imports and scheduled background work
File storage	MinIO or S3-compatible storage	Stores files such as photos and verification documents
Shared contracts	Shared TypeScript definitions and validation	Keeps different parts of the application consistent
Containers	Docker and Docker Compose	Packages and runs the application's services

The backend is organised into modules, including dealers, inventory, marketplace, buyers, search, notifications, and administration. These modules run within one backend application.

The worker runs separately so that large imports do not have to finish inside the original browser request.

Source: README.md.

4. How signing in and permissions work

Firebase handles the user's identity. MotorX stores its own application profile, including the user's role and account status.

A typical protected request follows these steps:

1. The user signs in through Firebase.
2. Firebase provides a token proving the user's identity.
3. The browser sends that token with a request.
4. The backend checks the token.
5. MotorX loads the local user record.
6. The backend checks whether the account is active.
7. It checks whether the user has the required role.
8. For dealer-owned information, it also checks ownership.

For example, being a dealer does not give someone permission to edit every dealer's listings. The requested listing must belong to that dealer.

The application also includes email-verification prompts and related account-interface handling. An email-verification banner should not, by itself, be described as proof that every backend operation requires a verified email.

5. How someone becomes a dealer

A person does not automatically receive dealer permissions just by filling out a registration form.

1. The person creates or signs into an account.
2. They submit a dealer application.
3. They provide business details and verification documents.
4. The application waits for administrator review.
5. An administrator approves or rejects it.
6. An approved applicant receives dealer access.
7. A rejected applicant can correct the application and submit it again.

Business information includes the business name, registration number, representative, location, contact information, dealership type, and description.

The current system preserves review history. This helps both parties understand earlier decisions and later resubmissions.

Approved dealers can update their public business profile. Buyers can see relevant business details when viewing a vehicle.

Source: `apps/backend/src/modules/dealers/dealer.service.ts`.

6. How vehicle listings work

A listing is the advertisement for one vehicle. It contains the vehicle registration number; title, make, model, and year; price and currency; location and description; category; photos; and current listing status.

MotorX has dedicated vehicle definitions for cars, motorcycles, vans, trucks, three-wheelers, and buses. The shared model also includes an “other” category.

Different vehicle types require different information. A car may need a body type, while a motorcycle needs a motorcycle type. Fuel type also affects validation. An electric vehicle requires battery information, while a petrol vehicle requires engine information.

This prevents every vehicle from being forced into the same unsuitable form.

Status	Meaning
Draft	Saved for the dealer to prepare or review
Active	Published for buyers to discover
Sold	Marked as sold
Archived	Removed from normal advertising while retaining the record

These statuses are controlled by backend rules. Not every transition is allowed in every situation. The current public visibility rules also exclude listings belonging to suspended dealers.

7. How dealers add and maintain vehicles

Dealers can add vehicles individually or through a bulk upload.

For a single vehicle, the dealer fills in the listing form, saves the record, adds photos, and manages its publication status.

The current dealer tools support editing existing listings, including descriptions, prices, and specifications. Dealers can also preview their own unpublished records, manage photos, and perform actions on multiple listings.

Examples of bulk actions include:

- Publishing eligible drafts.
- Marking active vehicles as sold.
- Archiving listings.
- Confirming that vehicles are still available.
- Reducing prices by a percentage.
- Deleting eligible archived listings.

These actions remain restricted to the dealer’s own inventory. The backend checks which selected records are eligible and returns counts showing what was updated or skipped.

8. How bulk CSV uploads work

CSV is a simple spreadsheet format. Each row represents a vehicle, and each column represents a field such as make, model, price, or registration number.

1. The dealer chooses a vehicle category.
2. They download the relevant template.
3. They fill in the vehicle information.
4. They upload the CSV.
5. MotorX stores the file and creates an upload-job record.
6. The job is placed in the processing queue.
7. The worker reads and processes the rows.
8. Valid records become listings.
9. Invalid or duplicate records are reported.
10. The dealer sees the final upload result.

The worker performs what is commonly called ETL:

Stage	Simple meaning
Extract	Read information from the uploaded file
Transform	Clean and standardise values, then validate them
Load	Save the accepted records into the database

For example, the worker can trim unnecessary spaces, standardise values, and interpret numbers before validating a row.

An upload can finish successfully, finish with some rejected rows, or fail. The job record stores progress and outcome counts so that the dealer can understand what happened.

9. How bulk photo uploads work

Vehicle information and vehicle photos are uploaded in separate steps. After the CSV creates the listings, the dealer can upload a ZIP file containing folders named after registration numbers.

For example, vehicle-photos.zip can contain a CAX-1234 folder with front.jpg and interior.jpg, and a WPB-5678 folder with front.jpg and rear.jpg.

MotorX matches these folders to the listings created by the corresponding inventory upload. The worker checks and processes the images, stores them, and attaches them to the correct vehicles.

This lets a dealer add photos for many vehicles without opening every listing individually.

The current implementation also handles retrying photo processing and reattaching photos to an existing upload, with checks to avoid conflicting work.

10. How buyers find vehicles

Buyers have several discovery options.

Structured filtering means selecting exact requirements, such as category, price range, make, model, year, fuel type, transmission, condition, or location.

Natural-language search lets the buyer type a more ordinary sentence, such as “Automatic car under 10 million.” The search system tries to recognise useful requirements from that text.

Hybrid search combines word matching with meaning-based ranking when the required search services and indexes are available.

- Word matching looks for the words in the request.
- Meaning-based matching helps rank vehicles related to the buyer's intention.
- Explicit filters still constrain which vehicles should qualify.

If the semantic search path is unavailable, the application has a lexical fallback, which means it can continue using text matching.

This does not mean that every search uses AI. Some requests are handled directly as structured searches.

Source: `apps/backend/src/modules/search/search.service.ts`.

11. Vehicle comparison and recommendations

The current comparison feature lets a buyer select **up to three vehicles** and compare their details together.

The selection is stored in that browser, so it can survive page changes and reloads without requiring an account. It is not an account-wide comparison list shared across devices.

The recommendation features provide two different experiences:

- Similar vehicles: alternatives related to the vehicle currently being viewed.
- Recommended vehicles: suggestions based on vehicles recently viewed in that browser.

The recommendation code considers characteristics such as vehicle category, make, model, price, and age.

Recently viewed vehicle IDs are sent to the backend for that recommendation request. The implementation does not create a server-side buyer browsing-history profile for this feature.

These are similarity-based recommendations; they should not be described as a separately trained personalised AI model.

12. What administrators do

Administrators manage the business operation of the marketplace. They can review dealer applications, inspect supporting documents, approve or reject applicants, manage account status, moderate listings, and inspect upload activity.

Audit records capture significant administrative actions. For example, when an administrator archives a listing, the audit record helps explain which administrator acted and which listing was affected.

Some administrative actions now save the business change and its audit record together in a database transaction. This means both changes succeed together or neither is committed.

The system also has backend health information for dependencies such as the database, queue, and worker. Operational health information is separate from proof that every business feature is working correctly.

13. Notifications and email

MotorX uses in-app notifications and email for selected events, including dealer application decisions, upload completion or failure, photo-processing problems, administrative listing removal, account suspension, and reminders about older inventory.

The notification centre supports unread counts and read state.

Email is now handled through an outbox. The application records that an email needs to be sent, and a background worker delivers it.

If the mail service is temporarily unavailable, the worker retries later. The dealer's original action does not need to wait for the mail server.

The outbox tracks whether delivery is pending, sent, or failed. A rare crash after sending but before recording success can produce a repeated email; the implementation does not promise exactly-once delivery.

14. How MotorX stores information

The database and file store have different responsibilities.

The database stores structured records: users, dealer profiles, listings, upload jobs, rejected rows, notifications, and administrative history.

File storage holds the actual photos, CSVs, ZIPs, and verification documents. For example, a listing record stores references to its images. The image bytes live in object storage.

This separation is useful because a vehicle description and a large photo are different kinds of data with different storage needs.

The backend and worker both access some of the same database collections. Changes to shared collection structures therefore need to remain compatible across both applications.

15. What already existed before August 27

The August 23 version already documented these capabilities:

- Firebase authentication and user roles.
- Dealer applications and approval.
- Multiple vehicle categories.
- Category-specific CSV templates and bulk inventory processing.
- Registration-number duplicate checks.
- ZIP photo attachment.
- Buyer browsing and filters.
- Dealer information on vehicle detail pages.
- Basic administration and moderation.
- Notifications.
- Natural-language and hybrid search.

These should be presented as the project's existing foundation. The work after August 27 extended that foundation, repaired incomplete behaviour, and added new features. It would be inaccurate to claim that all core capabilities were introduced during this later period.

16. Improvement: dealer application and profile workflows

The later work improved what happens throughout a dealer's application journey.

Rejected applicants can correct and resubmit their details. Earlier review information is preserved, and old decision fields are cleared appropriately when the application returns to review.

The application also gained a dedicated dealer profile page and profile-update workflow. This matters because a dealer's business information can change after approval. Contact numbers, descriptions, and other editable business details should not remain permanently tied to the original application.

The administrator side gained more complete application views and review-history information.

For example, a dealer is rejected because the submitted information is incomplete. They correct it and resubmit. The administrator can review the new submission while still understanding the earlier rejection.

This work appears particularly in the September 26 dealer-lifecycle changes.

17. Improvement: listing management

The later work completed important parts of maintaining inventory after initial creation.

- A route and form for editing listings.
- An owner-only detail endpoint for drafts and other non-public records.
- Fixes to description updates.
- Search-data regeneration using updated listing information.
- Photo retry behaviour after a listing has already been created.

- Better listing filters and counts.
- Bulk inventory actions.

One useful fix concerns partial success. If the listing is saved but uploading a photo fails, the interface can retain the listing identity and retry the photo operation. The dealer should not have to create the vehicle advertisement again.

The description fix also matters: saving an edited description must update the stored record and the information used for search.

18. Improvement: older inventory can be reconfirmed

Listings now include information about when the dealer last confirmed them. The shared stale-listing threshold is **60 days**. Background processing can remind dealers about stock that has not been confirmed within that period.

Dealers can explicitly confirm that a vehicle is still available.

This helps address a common marketplace problem: old advertisements remaining visible after the real situation has changed. The reminder is a request for the dealer to check their stock. It is not independent proof that the vehicle remains available.

19. Improvement: comparison, suggestions, and languages

The September 26 frontend work added comparison of up to three vehicles, similar-vehicle suggestions, recommendations based on recent viewing, English/Sinhala/Tamil interface dictionaries, a language switcher with a remembered preference, and more responsive layouts for smaller screens.

The language provider can also use the browser's preferred language for the initial choice when it recognises Sinhala or Tamil.

This improves access for a wider local audience. However, interface translation does not automatically translate free-text vehicle descriptions written by dealers.

The mobile changes include shared responsive layout components, navigation adjustments, and table handling.

The application also uses lazy loading for pages, allowing page code to load as needed. The code demonstrates this change, but a measured improvement in real mobile loading speed would require a separate browser performance test.

20. Improvement: database duplicate protection

Earlier duplicate checking relied on checking whether a registration number was already in use before inserting a listing. That can fail when two requests arrive at almost the same time:

1. Request A checks and sees no duplicate.
2. Request B checks and also sees no duplicate.
3. Both try to create the same registration.

The current model defines a database-level unique rule for normalised registration numbers in **draft or active** listings. This adds a stronger final check at the database.

Sold and archived records are excluded from that particular constraint, allowing legitimate resale while controlling duplicate live inventory.

There is also a unique rule linking an imported listing to its upload job and source row. This helps prevent retries from creating the same imported row more than once.

These protections are defined in `apps/backend/src/modules/marketplace/listing.model.ts`. Their deployment benefit depends on the corresponding indexes being created successfully in the target database.

21. Improvement: import recovery

This is one of the most significant technical changes. A background job can fail because a worker stops, Redis becomes unavailable, a network request fails, or the application restarts during deployment.

Protection	Simple explanation	Benefit
Durable job record	MongoDB remembers accepted work	The queue is not the only record of the task
Reconciliation	Background code looks for pending or abandoned work	Jobs that were not successfully queued can be queued again
Worker lease	A worker temporarily claims a job	Processing ownership can be tracked
Lease renewal	An active worker renews its claim	Long-running work can retain ownership
Ownership token checks	Updates must come from the current owner	An outdated worker cannot freely overwrite a newer attempt's job state
Checkpoints	Processing progress is recorded	A retry can resume from saved progress
Stable row and photo identities	Repeated processing recognises previous work	Retries are less likely to create duplicate data
Bounded retries	Temporary failures get limited additional attempts	A failing job does not retry forever

For example, if an upload is accepted while Redis is unavailable, its saved job record can remain pending. A later reconciliation cycle can dispatch it once the queue is usable.

Temporary failures and permanent input problems are also handled differently. A brief storage outage is worth retrying; an unreadable file generally needs correction.

Dealer-facing retry operations were added for failed inventory and image jobs. These mechanisms improve recovery. They do not replace testing actual crash and dependency-outage scenarios.

Source: `docs/RESILIENCE.md`.

22. Improvement: safer file handling

The upload changes add stronger checks around images, documents, and ZIP archives.

Images are decoded and rebuilt, which helps verify that the input is an actual readable image and removes metadata such as embedded location information.

Verification documents are checked using file content rather than trusting the filename or the type declared by the browser.

The PDF checks reject recognised active-content markers, such as scripting or embedded-file features. This is a limited content check, not a full antivirus scan.

ZIP processing gained limits and checks intended to control expanded file sizes, entry counts, invalid images, and repeated processing.

The work also improved practical upload behaviour, including wrapper folders, clearer empty-ZIP feedback, and retrying photo attachment.

A document-retention job was added to remove reviewed verification documents after a configured retention period. It removes stored files before clearing their database references, allowing failures to be retried.

23. Improvement: abuse controls

The backend now includes limits for different kinds of activity, including general requests, searches, dealer applications, CSV uploads, and photo uploads.

There are also daily upload quotas and a limit on how many file uploads one backend instance handles concurrently. These controls help prevent a small number of clients from exhausting resources.

For example, if the server is already handling its allowed number of simultaneous uploads, another upload can receive a controlled “busy” response instead of consuming additional upload memory.

Token verification also gained a bounded cache. This reduces repeated Firebase verification calls while respecting token expiry and a configured cache lifetime. Local account suspension remains checked separately.

24. Improvement: stronger moderation

Suspending an account now affects public listing visibility, including the relevant discovery queries. This means that suspension does more than block a dealer from editing their account while leaving all their advertisements normally discoverable.

The implementation caches suspended-dealer information briefly. It invalidates the local cache when the administrative status changes; other backend instances may take up to the short cache window to refresh.

Administrative account-status changes and listing archival also gained transactional audit recording. These changes improve both enforcement and traceability.

25. Improvement: tests are protected from real data

The earlier test setup could use the normal application database configuration for tests that clear collections.

The current test helper requires a separate `TEST_MONGODB_URI`. It checks that:

- The URI uses a MongoDB scheme.
- The database name follows the allowed `motorx_test` pattern.

- Remote hosts are rejected unless explicitly enabled for a disposable test database.
- The normal application URI is not silently used as a fallback.

This is a major data-safety improvement because automated tests often create and delete records.

The test environment also includes a MongoDB replica set, which supports testing operations that use database transactions.

Source: `apps/backend/src/test/db.ts`.

26. Improvement: broader automated quality checks

The repository now defines more extensive automated checks:

- Building the shared package, backend, worker, and frontend.
- Running backend, worker, and frontend tests.
- Checking dependency vulnerabilities.
- Validating Docker Compose configuration.
- Scanning Git history for secrets.
- Building application container images.
- Scanning those images for vulnerabilities.
- Scheduled checks and dependency-update configuration.

The project configuration was also updated to use Node.js 24 in the relevant runtime and build definitions.

Secret scanning has configured rules and historical exceptions, so its presence should not be described as proof that every historical credential problem has been resolved.

A checked-in workflow proves that the checks are defined. A particular release's CI result is needed to prove that they passed for that release.

Source: `.github/workflows/ci.yml`.

27. Improvement: deployment and operational support

The changes include improved health checks, worker heartbeat information, bounded shutdown handling, safer local port bindings, and additional deployment documentation.

The repository supports a single-server EC2 Compose configuration. Its override exposes the frontend and API while leaving Redis and MinIO bound privately through the base configuration.

A September 27 change allows the configured EC2 hostname in the Vite frontend server.

The repository also contains a separate ECS/Fargate deployment workflow and guide. These are different deployment arrangements. The presence of ECS documentation does not prove that the observed EC2 system runs on ECS, has multiple availability zones, or has managed failover.

Source: `compose.ec2.yml`.

28. Improvement: photo migration and thumbnails

The later work added image migration tooling and smaller thumbnail images. The purpose is to move older photo records into the newer storage arrangement and provide smaller images for places such as listing cards.

Smaller images can reduce the amount of image data required when displaying many vehicles.

The saved migration record reports:

- 90 images migrated.
- 1 already migrated.
- 78 missing source images.
- 0 migration failures.
- 90 thumbnails created.

The missing count matters. A migration can process files that exist, but it cannot recreate an original image that is absent from storage. Later live checks also found unresolved photo-loading problems, described below.

29. Timeline of recorded improvements

Date	Main recorded work
September 21	Admin workflows and filtering, safer test database setup, listing/upload changes, Docker and documentation updates, CI fixes, and dependency-audit work
September 23	Dealer dashboard improvements and worker-update fixes
September 25	Runtime, security, upload, job-reliability, and secret-scanning changes; dealer resubmission cleanup
September 26	Abuse controls, email outbox, dealer lifecycle and review workflows, mobile layouts, translations, recommendations, comparison, bulk listing tools, image migration, and expanded test evidence
September 26	Branch integration preserving dealer workflows and incorporating photo retry, ZIP, title, filter, and count fixes
September 27	EC2 hostname support and documentation updates

Useful commits for tracing these changes include 883a052, b1ff882, 7704608, 3c91252, 28e07d4, ffac7f9, 48c99be, and 44c1bdc.

30. What the saved test evidence proves

The September 26 test metadata reports:

Area	Recorded result
Backend tests	147 passed out of 147
Worker tests	94 passed out of 94
Frontend tests	50 passed out of 50
Workspace build	Completed successfully
Manual browser end-to-end procedures	Not executed at the time of that report

That is **291 recorded passing tests across those three suites**.

These saved results were read, not rerun during preparation of this explanation. They describe the recorded test run, not a fresh verification of every current working-tree change.

The import benchmark recorded 5,000 rows processed in approximately **11.1–11.2 seconds**, around **445–450 records per second**, against a stated 120-second target.

That is useful evidence for the tested local environment. It is not a guaranteed production speed for every file, database, or concurrent workload.

Sources: docs/test-evidence/run-metadata.json and docs/test-evidence/benchmark-output.txt.

31. What remains incomplete or needs careful wording

The saved September 27 EC2 smoke test recorded **15 passed checks and 2 failed checks**.

The passing checks included API health, basic browsing, a price filter, natural-language search handling, access checks for protected routes, dealer details, recommendation behaviour, and frontend route responses.

The two failures were:

- Main photos: 22 of 23 checked photos loaded; one failing reference used localhost:3000.
- Thumbnails: the one checked small-image reference did not load.

A localhost URL in a buyer's browser points to that buyer's own computer, which explains why such a stored image reference is unsuitable for a remotely hosted marketplace.

The frontend route checks confirm that the application shell was served. They do not prove that every interactive browser journey worked.

Source: docs/test-evidence/ec2-smoke-results.json.

There are also documentation inconsistencies. For example, the database document still describes registration uniqueness in terms of the earlier non-unique approach, while the current model defines a partial unique index. Some documentation links point to removed documents.

At the time of review, the working tree contained uncommitted report-generation and smoke-script changes. Those local changes should be distinguished from committed release history.

32. Plain-language presentation explanation

“MotorX is a vehicle marketplace with three main user groups: buyers, dealers, and administrators. Buyers can browse vehicles, search using filters or ordinary language, compare vehicles, and see recommendations. Dealers can apply for approval, maintain their business profile, and manage vehicles individually or through bulk spreadsheet and photo uploads. Administrators review dealers, manage accounts, moderate listings, and monitor important activity.

“The system has a website, a backend that applies rules, a database that stores records, file storage for photos and documents, and a separate worker that processes large tasks in the background.

“Since August 27, we have completed more of the dealer and administrator workflows, added comparison, recommendations, local-language support, mobile layouts, and bulk inventory tools. We have also strengthened duplicate prevention, upload validation, background-job recovery, email delivery, test-data protection, and automated quality checks.

“The saved automated test run reports 291 passing tests across the backend, worker, and frontend suites. The latest saved EC2 smoke check passed 15 checks and identified two remaining photo-loading issues that still need attention.”